

Module 5

Generative Modelling

Selina Ochukut

Module Overview

While previous modules focused on models that map an input to a label or structured output (*discriminative* modeling), this module studies **generative modelling**: which is learning to produce new data samples that plausibly could have come from the same distribution as a training dataset. This includes the **autoencoder**, a simple architecture for learning compressed representations that motivates, but ultimately falls short of, being a true generative model. We then develop the **Variational Autoencoder (VAE)**, which repairs the autoencoder's shortcomings using a principled probabilistic framework and the evidence lower bound. Next, you will explore **Generative Adversarial Networks (GANs)**, which replace explicit likelihood modeling with an adversarial game between two networks, and survey the most influential GAN variants. Finally, you will learn about **diffusion models**, the current state of the art for high-fidelity generation, and trace their surprising origin in **non-equilibrium thermodynamics**.

Expected Learning Outcomes

By the end of this module, you should be able to:

1. **Explain** the principles of generative modelling and distinguish it from discriminative modelling.
2. **Implement** autoencoders, Variational Autoencoders (VAEs), and Generative Adversarial Networks (GANs) to generate synthetic data and learn meaningful latent representations from real-world datasets.
3. **Analyze** the theoretical foundations, strengths, and limitations of autoencoders, VAEs, GANs, and diffusion models.

4. **Evaluate** the suitability of different generative modelling approaches for a given application.

1 Autoencoders

1.1 Generative vs. Discriminative Modeling

Every model studied in earlier modules – classifiers, detectors, segmentation networks – learns a mapping from input to some target output, implicitly or explicitly modeling a conditional distribution $p(y | x)$. A **generative model**, by contrast, aims to model the data distribution $p(x)$ itself (or a distribution $p(x | y)$ conditioned only on a coarse label, or nothing at all), so that new, realistic samples x can be drawn from it.

Key Idea

Learning $p(x)$ is a fundamentally harder problem than learning $p(y | x)$: a classifier only needs to find a decision boundary that separates classes, while a generative model must capture the full structure, variability, and correlations present in the data distribution well enough to produce convincing new examples. This added difficulty is precisely what motivates the increasingly sophisticated architectures studied in this module.

1.2 Video: Autoencoders

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/7mRfwaGGAPg>



1.3 Autoencoder Architecture

Definition

An **autoencoder** consists of two neural networks trained jointly:

- An **encoder** $f_\theta : x \mapsto z$, mapping an input $x \in \mathbb{R}^d$ to a lower-dimensional **latent code** (or **bottleneck representation**) $z \in \mathbb{R}^k$, typically with $k \ll d$.
- A **decoder** $g_\phi : z \mapsto \hat{x}$, mapping the latent code back to a reconstruction $\hat{x} \in \mathbb{R}^d$ of the original input.

The two networks are trained end-to-end to minimize a **reconstruc-**

tion loss between x and $\hat{x} = g_\phi(f_\theta(x))$, most commonly mean squared error $\|x - \hat{x}\|^2$ for continuous data, or binary cross-entropy for data normalized to $[0, 1]$.

Key Idea

The bottleneck (the low-dimensional latent space, with $k \ll d$) is the single most important design element of an autoencoder: without it, the network could trivially learn the identity function $g_\phi(f_\theta(x)) = x$ (e.g., simply copying the input through), learning nothing useful. Forcing information through a narrow bottleneck compels the encoder to discover a compressed representation that captures the most important, informative structure of the data – the defining goal of **unsupervised representation learning**. An autoencoder with $k \geq d$ is called **over-complete** and requires additional constraints (e.g., sparsity penalties, or the noise-based corruption used in denoising autoencoders below) to avoid learning a trivial identity mapping.

1.4 Connection to PCA

Example 1. Consider an autoencoder with a single linear layer for both the encoder and decoder (no nonlinearity), trained with mean-squared reconstruction error: $z = Wx$, $\hat{x} = W'z$. It can be shown that the optimal solution learned by such a linear autoencoder spans the same subspace as the top- k principal components found by **Principal Component Analysis (PCA)**. This gives a useful, concrete anchor for intuition: a (linear) autoencoder is a nonlinear generalization of PCA, learning a (possibly curved, nonlinear) low-dimensional manifold that best explains the data under reconstruction error, rather than being restricted to a linear subspace.

1.5 Denoising and Sparse Autoencoders

- **Denoising Autoencoder (DAE):** The encoder is given a *corrupted* version $\tilde{x}$ of the input (e.g., x with added Gaussian noise, or randomly zeroed-out pixels), but the reconstruction loss is computed against the *original, clean* x . This forces the network to learn robust features that capture the underlying data manifold rather than memorizing exact pixel values, and it also permits an overcomplete latent space (since perfectly copying the corrupted input would not minimize the loss).

- **Sparse Autoencoder:** An explicit sparsity penalty (e.g., an L_1 penalty on the latent activations, or a penalty encouraging the average activation of each latent unit to stay near some small target value) is added to the loss, encouraging most latent units to be inactive (near zero) for any given input, even when the latent dimension k is large.

1.6 Why Autoencoders Are Not (Good) Generative Models

A natural idea: once an autoencoder is trained, could we generate new samples by feeding a *random* latent vector z into the decoder g_ϕ ? In practice, this fails badly.

Key Idea

A standard autoencoder’s training objective (reconstruction loss alone) places **no constraint whatsoever on the structure of the latent space**. The encoder is free to map training examples to arbitrary, scattered, or disconnected regions of latent space, as long as the decoder can reconstruct them – there is no pressure for the latent space to be smooth, continuous, or well-covered. Consequently:

- There may be large “holes” in latent space that no training example maps to; decoding a randomly sampled z from such a region typically produces unrealistic, garbled output.
- Even interpolating between the latent codes of two real, valid inputs may pass through regions the decoder has never learned to handle well, producing blurry or nonsensical intermediate reconstructions.
- There is no specified *prior distribution* over z to sample from in the first place – we would have to arbitrarily guess a plausible range or distribution for latent vectors, with no guarantee it matches where the encoder actually placed the data.

This is precisely the gap that Variational Autoencoders are designed to close: by explicitly regularizing the latent space to follow a known, fixed prior distribution (typically a standard Gaussian) from which new samples can be validly and reliably drawn.

2 Variational Autoencoders (VAEs)

2.1 Video: Variational Autoencoders

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/fcvYpzHmhvA>



The **Variational Autoencoder** (Kingma & Welling, 2013) reformulates the autoencoder as a proper probabilistic generative model, trained via principled approximate Bayesian inference, directly solving the latent-space regularization problem identified in Section 1.6.

2.2 The Probabilistic Generative Model

Definition

A VAE assumes data x is generated by the following process: first, a latent variable z is drawn from a fixed **prior distribution** $p(z)$ (typically $\mathcal{N}(0, I)$); then, x is drawn from a **likelihood** (decoder) distribution $p_\theta(x | z)$, parameterized by a neural network. The marginal likelihood of the data is:

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz$$

Key Idea

If we could compute $p_\theta(x)$ and its gradient, we could train the decoder by direct maximum likelihood. Unfortunately, this integral is intractable for any reasonably expressive neural network decoder – it would require integrating over the entire (typically continuous, high-dimensional) latent space. Similarly, the true posterior $p_\theta(z | x) = \frac{p_\theta(x|z)p(z)}{p_\theta(x)}$, which we would need in order to infer a good latent code for a given x , is intractable for the same reason. VAEs solve both problems simultaneously by introducing an **approximate posterior** $q_\phi(z | x)$ (the encoder), and optimizing a tractable *lower bound* on $\log p_\theta(x)$ instead of the intractable quantity itself.

2.3 Deriving the Evidence Lower Bound (ELBO)

Definition

For any choice of approximate posterior $q_\phi(z | x)$, the following identity holds exactly:

$$\log p_\theta(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)] - D_{KL}(q_\phi(z | x) \| p(z))}_{\text{ELBO}(\theta, \phi; x)} + D_{KL}(q_\phi(z | x) \| p_\theta(z | x))$$

Since KL divergence is always non-negative, the **Evidence Lower Bound (ELBO)** satisfies $\text{ELBO}(\theta, \phi; x) \leq \log p_\theta(x)$, with equality exactly when $q_\phi(z | x)$ matches the true (intractable) posterior $p_\theta(z | x)$.

Key Idea

This identity is the crux of the entire VAE framework: since the ELBO is a *lower bound* on the (intractable) data log-likelihood, and the ELBO itself is fully tractable (both terms can be computed and differentiated), we can train the model by *maximizing the ELBO* with respect to both the decoder parameters θ and the encoder parameters ϕ . Maximizing the ELBO simultaneously (a) increases (a lower bound on) how well the model explains the data, and (b) tightens the bound by pushing $q_\phi(z | x)$ closer to the true posterior – so a good encoder and a good decoder are learned jointly, using a single unified objective.

2.4 Interpreting the ELBO's Two Terms

The VAE training objective, for a single data point x , is (equivalently, taking the negative to get a loss to minimize):

$$\mathcal{L}(\theta, \phi; x) = - \underbrace{\mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x | z)]}_{\text{reconstruction term}} + \underbrace{D_{KL}(q_\phi(z | x) \| p(z))}_{\text{regularization term}}$$

- **Reconstruction term:** Encourages the decoder, given a latent code sampled from the encoder's approximate posterior, to accurately reconstruct the original input – exactly analogous to the standard autoencoder's reconstruction loss from Section 1.
- **Regularization (KL) term:** Encourages the encoder's approximate posterior $q_\phi(z | x)$ to stay close to the fixed prior $p(z) = \mathcal{N}(0, I)$ for *every*

input x . This is precisely the missing ingredient identified in Section 1.6: it directly forces the latent space to be well-organized around a known distribution, with no “holes,” so that sampling $z \sim \mathcal{N}(0, I)$ and decoding it reliably produces realistic outputs.

2.5 The Reparameterization Trick

A key implementation challenge: to train by backpropagation, we need gradients of the ELBO with respect to ϕ , but the reconstruction term involves an expectation over $z \sim q_\phi(z | x)$ – i.e., we need to differentiate *through a random sampling operation*, which is not directly possible.

Definition

The encoder is designed to output the parameters of a distribution – typically a diagonal Gaussian, $q_\phi(z | x) = \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$ – rather than a sample directly. The **reparameterization trick** rewrites sampling $z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$ as a deterministic, differentiable function of the encoder’s outputs and an independent noise source:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

where $\odot$ denotes elementwise multiplication and ϵ is sampled independently of any learnable parameters.

Key Idea

By moving all the “randomness” into ϵ (which does not depend on θ or ϕ), the sampling operation becomes a simple, differentiable arithmetic expression in terms of $\mu_\phi(x)$ and $\sigma_\phi(x)$. Gradients can now flow from the reconstruction loss, through the sampled z , back into the encoder’s parameters ϕ via ordinary backpropagation – exactly as they would through any other differentiable layer. This trick is what makes end-to-end training of the whole VAE (encoder and decoder jointly) possible with standard gradient-based optimizers.

2.6 Closed-Form KL Term for Gaussians

When both $q_\phi(z | x) = \mathcal{N}(\mu, \sigma^2 I)$ and the prior $p(z) = \mathcal{N}(0, I)$ are Gaussian, the KL divergence term has a simple closed form (per latent dimension j ,

summed over all k dimensions):

$$D_{KL}(q_\phi(z | x) \| p(z)) = \frac{1}{2} \sum_{j=1}^k \left(\mu_j^2 + \sigma_j^2 - \log(\sigma_j^2) - 1 \right)$$

Example 2. Suppose a 1-dimensional latent code has $\mu = 2.0$, $\sigma^2 = 0.5$. The KL penalty for this dimension is:

$$\frac{1}{2} (2.0^2 + 0.5 - \log(0.5) - 1) = \frac{1}{2} (4 + 0.5 + 0.693 - 1) = \frac{1}{2} (4.193) \approx 2.097$$

This penalty is minimized (equal to zero) exactly when $\mu = 0$ and $\sigma^2 = 1$ – i.e., when the encoder’s output distribution for this input exactly matches the standard normal prior – and grows as μ moves away from 0 or σ^2 moves away from 1 in either direction, directly enforcing the desired latent-space regularization.

2.7 Generating New Samples

Once trained, generating new data with a VAE is simple: sample $z \sim \mathcal{N}(0, I)$ directly from the prior (no encoder needed at generation time), and pass it through the trained decoder $p_\theta(x | z)$ to obtain a new sample $\hat{x}$.

2.8 The Beta-VAE and the Reconstruction-Regularization Trade-off

Key Idea

In practice, a tension exists between the two ELBO terms: strongly minimizing the KL term pushes every input’s latent code toward the same generic prior distribution, which can cause the decoder to ignore fine-grained information in z and produce blurry reconstructions, while over-prioritizing reconstruction can leave the latent space poorly regularized. The **β -VAE** introduces a tunable weight β on the KL term: $\mathcal{L} = -\mathbb{E}_q[\log p_\theta(x | z)] + \beta \cdot D_{KL}(q_\phi(z | x) \| p(z))$. Larger β (> 1) more strongly enforces the prior, often encouraging a more **disentangled** latent space (where individual latent dimensions correspond to independent, interpretable factors of variation in the data), at some cost to reconstruction fidelity; $\beta = 1$ recovers the standard VAE.

3 GANs and GAN Variants

Generative Adversarial Networks (Goodfellow et al., 2014) take a radically different approach from VAEs: rather than explicitly modeling and optimizing a likelihood (even approximately), a GAN trains a generative model implicitly, by setting up an adversarial game between two competing networks.

3.1 Video: Generative Adversarial Networks

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/ye5P78eK0VM>



3.2 The Adversarial Framework

Definition

A GAN consists of two networks trained simultaneously in competition:

- A **Generator** $G_\theta : z \mapsto x$, mapping a random noise vector $z \sim p(z)$ (e.g., $\mathcal{N}(0, I)$) to a synthetic sample $G_\theta(z)$, trying to produce samples indistinguishable from real data.
- A **Discriminator** $D_\phi : x \mapsto [0, 1]$, a binary classifier trying to distinguish real data samples from the generator's synthetic ("fake") samples.

The two are trained via the following two-player minimax game:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))]$$

Key Idea

The discriminator is trained to *maximize* $V(D, G)$: assign a high probability $D(x) \rightarrow 1$ to real data and a low probability $D(G(z)) \rightarrow 0$ to generated data. The generator is trained to *minimize* $V(D, G)$: produce samples $G(z)$ that fool the discriminator into assigning $D(G(z)) \rightarrow 1$. Crucially, the generator *never sees real data directly* – it only ever

receives a training signal indirectly, through the discriminator’s gradients, learning purely from the discriminator’s feedback about which of its samples were and were not convincing.

3.3 Training Procedure

In practice, D and G are updated in alternation (not via a single joint optimization), typically:

1. **Discriminator step:** Sample a batch of real data and a batch of generator samples (with G ’s parameters held fixed); update ϕ to increase $V(D, G)$ (better distinguish real from fake).
2. **Generator step:** Sample a batch of noise vectors z ; update θ to decrease $V(D, G)$ (produce samples the current discriminator finds more convincing).

Key Idea

In practice, the original minimax generator loss $\log(1 - D(G(z)))$ provides very weak gradients early in training, when the discriminator easily distinguishes fake from real (since $D(G(z)) \approx 0$, this term saturates). A standard practical fix is to instead train the generator to *maximize* $\log D(G(z))$ directly (the “non-saturating” loss) – driving the same qualitative behavior (fool the discriminator) but with much stronger gradients when the generator is performing poorly.

3.4 The Theoretical Optimum: Jensen-Shannon Divergence

Goodfellow et al. showed that, for a fixed generator, the optimal discriminator is $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$ (where p_g is the distribution induced by the generator), and that with this optimal discriminator, the generator’s objective reduces to minimizing (a shifted, scaled version of) the **Jensen-Shannon (JS) divergence**:

$$V(D^*, G) = 2 \cdot D_{JS}(p_{data} \parallel p_g) - \log 4$$

Key Idea

This result establishes what GAN training is theoretically doing: at the global optimum (an idealized discriminator perfectly tracking the current generator), the generator is driven to minimize the JS divergence between the real data distribution and its own generated distribution, which is minimized (equal to zero) exactly when $p_g = p_{data}$. In practice, the discriminator is a finite neural network trained with a finite number of gradient steps, not the theoretically optimal D^* , so real GAN training only approximates this idealized picture – and much of the practical difficulty of training GANs stems from this gap between theory and practice.

3.5 Training Instabilities

GANs are notoriously difficult to train reliably, due to several well-documented failure modes:

- **Mode collapse:** The generator discovers a small number of outputs (or even a single output) that reliably fool the current discriminator, and collapses to producing only these, failing to capture the full diversity of the data distribution.
- **Vanishing gradients:** If the discriminator becomes too good too quickly (perfectly separating real from fake), $D(G(z)) \rightarrow 0$ for all generator outputs, and the generator’s gradient signal vanishes, halting learning.
- **Non-convergence / oscillation:** Because the min and max players in the minimax game are updated in alternation rather than jointly, training can oscillate rather than converge to the theoretical equilibrium, especially when G and D ’s relative capacities/learning rates are poorly balanced.

3.6 GAN Variants

3.6.1 DCGAN (Deep Convolutional GAN)

DCGAN (Radford et al., 2015) established a set of architectural conventions for using convolutional networks reliably in both G and D : replacing pooling with strided convolutions (in D) and fractionally-strided/transposed convolutions (in G), using batch normalization in both networks, and avoiding fully-connected hidden layers – guidelines that stabilized GAN training on image data and became a template for later convolutional GAN architectures.

3.6.2 Conditional GAN (cGAN)

A **Conditional GAN** (Mirza & Osindero, 2014) conditions both G and D on auxiliary information y (e.g., a class label): $G(z, y)$ and $D(x, y)$. This allows controllable generation – e.g., specifying which digit class to generate – rather than sampling an unconditional, uncontrolled output.

3.6.3 Wasserstein GAN (WGAN)

Key Idea

WGAN (Arjovsky et al., 2017) replaces the JS-divergence-based objective with the **Wasserstein (Earth Mover’s) distance**, which remains well-behaved (provides useful gradients) even when the real and generated distributions have little or no overlap in their support – a common situation early in training that causes the JS divergence (and its gradients) to become uninformative. The discriminator is reframed as a “critic” that outputs an unbounded real-valued score (rather than a probability), and must satisfy a 1-Lipschitz constraint, originally enforced via **weight clipping** and later, more effectively, via a **gradient penalty** (WGAN-GP) that directly penalizes deviation of the critic’s gradient norm from 1. WGAN’s loss also correlates much better with actual perceptual sample quality, making training progress far easier to monitor than with the original GAN’s loss.

3.6.4 CycleGAN

CycleGAN (Zhu et al., 2017) performs **unpaired** image-to-image translation (e.g., converting horse photos to zebra photos) without requiring matched pairs of images in the two domains. It trains two generators ($G : X \rightarrow Y$ and $F : Y \rightarrow X$) and two discriminators, and introduces a **cycle-consistency loss** $\|F(G(x)) - x\|_1 + \|G(F(y)) - y\|_1$, which enforces that translating an image to the other domain and back should approximately recover the original – providing a training signal even without any paired ground-truth examples.

3.6.5 StyleGAN

StyleGAN (Karras et al., 2019) introduced a **style-based generator** architecture: rather than feeding the latent code z directly into the first generator layer, an intermediate “style” vector w (obtained by passing z through a learned mapping network) is injected at *every* resolution level of the generator via adaptive instance normalization, giving fine-grained, disentangled

control over different levels of visual detail (e.g., coarse pose at low resolution, fine texture at high resolution) and enabling high-fidelity, controllable image synthesis.

3.7 VAE vs. GAN: Summary Comparison

	VAE	GAN
Training objective	Explicit (approximate) likelihood maximization via ELBO	Implicit, adversarial min-max game (no explicit likelihood)
Training stability	Generally stable, well-behaved optimization	Can be unstable (mode collapse, vanishing gradients)
Sample quality	Often somewhat blurry (due to reconstruction term)	Often sharper, more perceptually realistic samples
Latent space	Explicitly regularized, structured	Less directly regularized; structure emerges implicitly
Likelihood evaluation	Can (approximately) evaluate data point likelihood	Cannot directly evaluate likelihood of a data point

4 Diffusion and Thermodynamic Generative Modelling

Diffusion models are the current state of the art for high-fidelity image, audio, and video generation. Remarkably, their foundational idea did not originate in machine learning, but was explicitly imported from **non-equilibrium statistical thermodynamics** (Sohl-Dickstein et al., 2015): the same mathematics that describes a physical system relaxing toward thermal equilibrium is used to describe data being progressively destroyed by noise, and generation is framed as *learning to run this physical process in reverse*.

4.1 Video: Diffusion Models

Video Visit the URL below to view a video:

<https://www.youtube.com/embed/3qLq-4Tmd3g>



4.2 The Forward (Noising) Process

Definition

The **forward diffusion process** is a fixed (non-learned) Markov chain that gradually adds a small amount of Gaussian noise to a data sample $x_0 \sim p_{data}$ over T discrete timesteps:

$$q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I), \quad t = 1, \dots, T$$

where $\beta_t \in (0, 1)$ is a small, fixed (or scheduled) **noise variance** at step t . Using the reparameterization trick and the properties of Gaussians, the marginal distribution of x_t given x_0 (skipping all intermediate steps) has a convenient closed form:

$$q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I), \quad \alpha_t := 1 - \beta_t, \quad \bar{\alpha}_t := \prod_{s=1}^t \alpha_s$$

Key Idea

As t increases, $\bar{\alpha}_t \rightarrow 0$ (given a suitable noise schedule), so x_t 's distribution approaches $\mathcal{N}(0, I)$ regardless of the original data x_0 – the forward process gradually erases all information about the original sample, converging to pure, structureless Gaussian noise. This is directly analogous to a physical system evolving toward its maximum-entropy thermal equilibrium state: just as a drop of ink diffuses through water until it is uniformly (and irreversibly, from a practical standpoint) spread out, the forward process diffuses a structured data distribution into a simple, fixed, high-entropy Gaussian distribution. This origin in physical diffusion processes is the literal source of the model family's name.

4.3 The Reverse (Denoising) Process

The generative model is the *reverse* of the forward process: starting from pure noise $x_T \sim \mathcal{N}(0, I)$, we want to iteratively remove noise, step by step, to recover a realistic data sample x_0 .

Definition

Since the true reverse conditional $q(x_{t-1} | x_t)$ is intractable in general (it technically depends on the entire data distribution), we learn an approximation using a neural network:

$$p_\theta(x_{t-1} | x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

Generation proceeds by sampling $x_T \sim \mathcal{N}(0, I)$ and then repeatedly sampling $x_{t-1} \sim p_\theta(x_{t-1} | x_t)$ for $t = T, T-1, \dots, 1$, finally producing x_0 .

Key Idea

Crucially, if each forward noising step β_t is small enough, the true reverse step $q(x_{t-1} | x_t)$ is well-approximated by a Gaussian, which is exactly the functional form we assumed for p_θ above – this is why the forward process uses *many small* noise steps rather than a single large one: it makes the reverse transitions tractable to model with a simple Gaussian, at the cost of requiring many steps (often hundreds to thousands in early diffusion models) to fully generate a sample.

4.4 Training Objective: A Hierarchical VAE

A diffusion model can be viewed as a special case of a (deep, hierarchical) VAE: $x_1, \dots, x_T$ play the role of a sequence of latent variables, and the fixed forward process plays the role of the (non-learned, fixed-form) approximate posterior q . Applying the same ELBO derivation as in Section 2 to this hierarchical setting yields a training objective that decomposes into a sum of KL divergence terms, one per timestep, each comparing the learned reverse step to the (tractable, closed-form) *forward posterior conditioned on* x_0 , $q(x_{t-1} \mid x_t, x_0)$.

4.5 Simplified Training: Predicting the Noise (DDPM)

Ho et al. (2020), in the **Denoising Diffusion Probabilistic Model (DDPM)** paper, showed that this seemingly complex objective simplifies enormously with the right parameterization.

Key Idea

Rather than having the network directly predict the mean $\mu_\theta(x_t, t)$ of the reverse step, DDPM reparameterizes the network to predict the **noise** ϵ that was added to produce x_t from x_0 in the forward process. Since $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$ for some $\epsilon \sim \mathcal{N}(0, I)$, predicting ϵ from x_t (and the timestep t) is equivalent, via simple algebra, to predicting μ_θ . With this reparameterization, the (re-weighted) training objective reduces to an extremely simple mean-squared-error loss:

$$L_{\text{simple}}(\theta) = \mathbb{E}_{x_0, \epsilon \sim \mathcal{N}(0, I), t} \left[\|\epsilon - \epsilon_\theta(x_t, t)\|^2 \right]$$

i.e., sample a real data point x_0 , a random timestep t , and random noise ϵ ; construct the noised sample x_t ; and train a single network ϵ_θ to predict which noise was added, given the noisy sample and the timestep.

Example 3. Suppose $\bar{\alpha}_t = 0.36$ at some timestep t , and a training image x_0 is corrupted with a sampled noise vector ϵ to produce $x_t = \sqrt{0.36} x_0 + \sqrt{1 - 0.36} \epsilon = 0.6 x_0 + 0.8 \epsilon$. The network ϵ_θ receives (x_t, t) as input and is trained (via ordinary gradient descent on the squared error between its output and the true ϵ) to recover the specific noise vector ϵ that was used – a simple, direct, and highly stable supervised regression task, in stark contrast to the adversarial minimax game required for GAN training.

4.6 Score-Based Generative Models and the Connection to Physics

An illuminating, mathematically equivalent perspective (Song & Ermon, 2019; Song et al., 2021) frames diffusion models in terms of the **score function** $\nabla_x \log p(x)$ – the gradient of the log-density of the data distribution.

Definition

Learning to predict the added noise $\epsilon_\theta(x_t, t)$ is, up to a known scaling factor, equivalent to learning the score function of the noised data distribution $p_t(x_t)$ at each noise level t :

$$\nabla_{x_t} \log p_t(x_t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}}$$

Generation can then be understood as running **Langevin dynamics** – a classical technique from statistical physics for sampling from a distribution given only its score function – at each noise level:

$$x \leftarrow x + \frac{\eta}{2} \nabla_x \log p(x) + \sqrt{\eta} \xi, \quad \xi \sim \mathcal{N}(0, I)$$

where η is a small step size. Iterating this update moves samples, on average, toward regions of higher data density, while the injected noise ξ prevents collapse to a single mode and ensures the correct target distribution is sampled from in the long run.

Key Idea

Langevin dynamics is itself a foundational tool from statistical physics, originally developed to describe the motion of a particle under both a deterministic force (here, the score, pulling toward high-density regions – analogous to a potential energy gradient) and random thermal fluctuations (here, the injected noise ξ , analogous to molecular collisions in a fluid at a given temperature). This is not a superficial analogy: the mathematics of score-based diffusion sampling *is* the mathematics of a particle undergoing Brownian motion in a potential landscape defined by $-\log p(x)$, placing diffusion models on a rigorous footing shared with statistical mechanics.

4.7 Diffusion as Stochastic Differential Equations

Song et al. (2021) showed that the discrete-time forward and reverse processes above are both discretizations of continuous-time **stochastic differential equations (SDEs)**:

$$\text{Forward SDE: } dx = f(x, t) dt + g(t) dw$$

$$\text{Reverse-time SDE: } dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{w}$$

where w is a standard Wiener process (Brownian motion) and $\bar{w}$ is a Wiener process running backward in time. This reverse-time SDE result is a direct application of a classical theorem (Anderson, 1982) on the time-reversal of diffusion processes, itself a foundational result in the stochastic-thermodynamics literature.

Key Idea

This continuous-time view unifies DDPM (a particular discretization choice) with score-based models under a single mathematical framework, and directly exposes the model family’s roots in the physics of diffusion: the forward SDE is mathematically identical in form to the Fokker-Planck description of a physical particle diffusing under Brownian motion, and generation amounts to exactly reversing this physical diffusion process using a learned score function – something only possible in expectation, using a model, since a real physical diffusion process cannot literally be run backward in time.

4.8 Thermodynamic Origins and Physical Computing

Key Idea

The very first diffusion model paper (Sohl-Dickstein et al., 2015, “*Deep Unsupervised Learning using Nonequilibrium Thermodynamics*”) drew its central idea directly from **non-equilibrium statistical mechanics**: the forward process is framed explicitly as a physical system being driven away from a complex, structured state and toward thermal equilibrium (maximum entropy) via a sequence of small, reversible-in-principle perturbations, closely mirroring techniques such as **simulated annealing** and **annealed importance sampling**, both of which likewise bridge a simple, easy-to-sample distribution and a complex target distribution via a sequence of intermediate distributions.

Two deeper thermodynamic connections are worth noting for a complete picture:

- **Entropy production and fluctuation theorems:** Each step of the forward noising process can be shown to increase the system’s entropy (in the thermodynamic, not just information-theoretic, sense), and the KL-divergence terms comprising the diffusion training objective have a direct interpretation in terms of the (average) entropy produced by the forward process versus the entropy the learned reverse process must “remove” to successfully denoise – connecting diffusion model training to fluctuation theorems (such as the Jarzynski equality) from stochastic thermodynamics, which relate the work done on a non-equilibrium system to free energy differences.
- **Physical (analog) thermodynamic computing:** A very recent and active research direction asks whether the stochastic, noise-driven dynamics used in diffusion sampling could be implemented directly in physical hardware that exploits real thermal noise, rather than being simulated step-by-step on conventional digital processors. Since Langevin dynamics and diffusion sampling are mathematically a description of a physical system relaxing under noise, specialized **thermodynamic computing** hardware (analog circuits whose natural thermal fluctuations directly realize the required stochastic dynamics) has been proposed as a potentially far more energy- and time-efficient substrate for running diffusion-style sampling and optimization, compared to explicitly simulating Gaussian noise addition on a digital GPU.

Key Idea

The throughline across this entire section is that diffusion models are not merely *inspired by* thermodynamics as a loose metaphor: their forward process, training objective, and sampling procedure are each directly derived from, and mathematically identical in form to, established results in non-equilibrium statistical physics (diffusion/Fokker-Planck equations, Langevin dynamics, time-reversal of stochastic processes, and entropy-production identities). This is a rare and notable case in modern deep learning where a highly successful architecture is best understood as an application of pre-existing physical theory, rather than an architecture designed from empirical trial-and-error first.

4.9 Comparison: Diffusion Models vs. VAEs and GANs

	VAE	GAN	Diffusion Model
Training objective	ELBO (approximate likelihood)	Adversarial min-max game	Simple denoising MSE
Training stability	Stable	Can be unstable	Very stable
Sample quality	Often blurry	Often sharp, but mode collapse risk	State of the art, sharp
Generation speed	Fast (single pass)	Fast (single pass)	Slow (many steps)
Theoretical grounding	Variational inference	Game theory / f -divergence	Thermodynamics, SDEs

REFERENCES

- Prince, Simon JD. Understanding deep learning. MIT press, 2023.
- Goodfellow, I., Bengio, Y., Courville, A., & Bengio, Y. (2016). Deep learning (Vol. 1, No. 2, pp. 1-800). Cambridge: MIT press. <https://www.deeplearningbook.org/>
- Bishop, Christopher M., and Hugh Bishop. Deep learning: Foundations and concepts. Vol. 1. Cham, Switzerland: Springer, 2024.
- Torralba, A., Isola, P., & Freeman, W. T. (2024). Foundations of computer vision. MIT Press.